Rydo

Intercity Cab Booking Platform

Project Documentation

Developed by: Mantej

Technology: Django (Python) | HTML | CSS | JavaScript | SQLite | Google Maps API

Purpose: Placement Portfolio Project

Year: 2026

1. Project Overview

1.1 Introduction

Rydo is a full-stack intercity cab booking web application built using Django (Python). The platform connects customers who need intercity travel from Chandigarh to major cities like Delhi, Shimla, Amritsar, Manali, Jammu, Srinagar, and Jaipur — and vice versa — with a fleet of verified drivers.

The project consists of two integrated websites running under the same Django project:

- **Customer Website:** Where passengers book cabs, view fares, and track their ride in real-time.
- **Driver/Admin Portal:** Where drivers receive live booking alerts, accept rides, and share GPS location. Admins monitor all bookings, manage drivers, and view revenue reports.

1.2 Problem Statement

Traditional intercity cab bookings in smaller cities like Chandigarh rely on phone calls, WhatsApp messages, or walk-in bookings. This creates problems like:

- No transparency in pricing — customers are often quoted different fares.
- No real-time tracking — family members cannot know where the cab is.
- No structured system for drivers to receive and manage bookings.
- No centralized revenue tracking for the cab company owner/admin.

Rydo solves all of these problems through a digital platform.

1.3 Objectives

- Build a complete online cab booking system for intercity routes from Chandigarh.
- Provide transparent, fixed fare pricing with no surge.
- Implement real-time GPS tracking of the driver on Google Maps.
- Create a live alert system for drivers so they can accept rides instantly.
- Give the admin a full dashboard with revenue, commission, and driver performance data.

2. Technology Stack

Layer	Technology	Purpose
Backend Framework	Django 4.2 (Python)	Core web framework — handles routing, views, ORM, authentication
Programming Language	Python 3.13	Backend logic, business rules, API endpoints
Frontend	HTML5 + CSS3 + Vanilla JS	Templates, styling, client-side interactivity
Database	SQLite	Local development database (zero-config, built into Python)
ORM	Django ORM	Database queries without raw SQL
Maps & GPS	Google Maps JavaScript API	Real-time driver location tracking on map
Authentication	Django Auth System	Login/logout for drivers and admins
Static Files	WhiteNoise	Serving CSS/JS in production
Admin Panel	Django Admin	Managing drivers, bookings, users
Fonts	Google Fonts (Nunito, Bebas Neue)	Typography for UI

3. Project Structure

The project follows Django's recommended app-based architecture. Each app handles a distinct domain of the application:

```
Rydo/
├── manage.py                  ← Django management commands
├── settings.py                ← Project configuration
├── project_urls.py            ← Root URL routing
├── requirements.txt           ← Python dependencies
│
├── cabs/                      ← Customer-facing app
│   ├── models.py              ← Database models (Booking, Driver, Contact)
│   ├── views.py               ← Page views + REST API endpoints
│   ├── forms.py               ← BookingForm, TrackForm, ContactForm
│   ├── urls.py                ← URL patterns for customer site
│   └── admin.py               ← Django admin configuration
│
├── driver_portal/             ← Driver & Admin portal app
│   ├── views.py               ← Dashboard, bookings, revenue, alerts
│   └── urls.py                ← URL patterns for portal
│
├── templates/
│   ├── cabs/                  ← 7 customer-facing HTML templates
│   └── driver_portal/         ← 6 portal HTML templates
│
└── static/
    ├── css/style.css          ← Customer site stylesheet
    └── css/portal.css         ← Driver/admin portal stylesheet
```

4. Database Design

The application uses 3 database models defined in cabs/models.py. Django ORM automatically creates SQLite tables from these Python classes.

4.1 Driver Model

Stores all information about cab drivers registered on the platform.

Field	Type	Description
driver_id	CharField	Auto-generated unique ID (e.g. DRV-1234)
user	OneToOneField → User	Links to Django auth user for portal login
name	CharField	Full name of the driver
phone	CharField	Contact number
license_number	CharField	Unique driving license number
vehicle_number	CharField	Unique vehicle registration number
vehicle_type	CharField (choices)	sedan / suv / tempo
vehicle_model	CharField	e.g. Innova Crysta, Swift Dzire
is_available	BooleanField	True = free, False = on a trip
is_active	BooleanField	False = driver is deactivated
current_lat	FloatField	Latest GPS latitude (updated every 10 sec)
current_lng	FloatField	Latest GPS longitude (updated every 10 sec)
last_location_update	DateTimeField	Timestamp of last GPS push
joined_date	DateField	When driver joined the platform

Key Methods:

- **total_rides():** Returns count of completed trips for this driver.
- **total_earnings():** Returns sum of fares from all completed trips.
- **save() override:** Auto-generates driver_id like DRV-1234 before first save.

4.2 Booking Model

Central model — every cab booking made on the platform is stored here.

Field	Type	Description
booking_id	CharField	Auto-generated (e.g. 1313-AB1234)
customer_name	CharField	Passenger full name
customer_phone	CharField	Used for booking lookup & tracking
customer_email	EmailField	Optional email
pickup_city	CharField (choices)	One of 8 cities
drop_city	CharField (choices)	One of 8 cities
pickup_date	DateField	Date of travel
pickup_time	TimeField	Time of pickup
cab_type	CharField (choices)	sedan / suv / tempo
passengers	PositiveIntegerField	Number of passengers
trip_type	CharField (choices)	one_way or round_trip
return_date	DateField (nullable)	Only for round trips
estimated_fare	DecimalField	Calculated fare in INR
commission_rate	DecimalField	Admin commission % (default 15%)
status	CharField (choices)	pending / confirmed / on_trip / completed / cancelled
driver	ForeignKey → Driver	Assigned driver (null until accepted)
accepted_at	DateTimeField	When driver accepted the booking
completed_at	DateTimeField	When trip was marked complete
created_at	DateTimeField	When customer placed the booking

Key Methods:

- **driver_earning():** Returns 85% of fare (after 15% commission deduction).
- **admin_commission():** Returns 15% of fare as platform commission.
- **save() override:** Auto-generates booking_id like 1313-AB1234 before first save.

4.3 ContactMessage Model

Stores messages submitted through the Contact Us form on the customer website.

Field	Type	Description
name	CharField	Sender name
email	EmailField	Sender email
phone	CharField	Sender phone (optional)
message	TextField	The message content

Field	Type	Description
created_at	DateTimeField	When message was submitted

4.4 Database Relationships

Driver → Booking: One-to-Many. One driver can have many bookings. A booking has at most one driver (ForeignKey).

User → Driver: One-to-One. Each driver has exactly one Django login account linked via OneToOneField.

5. Features — Customer Website

5.1 Homepage

- Quick booking widget with city dropdown, date picker, and Search Cabs button.
- Swap button to interchange pickup and drop cities.
- 7 popular route cards with distance and travel time.
- "Why Choose Rydo" section — 6 feature highlights.
- Fleet section showing Sedan, SUV, and Tempo Traveller with specs.
- Sticky navbar with mobile hamburger menu.

5.2 Cab Booking (book_cab view)

- Full booking form: customer details, route, date/time, cab type, passengers, trip type.
- Real-time AJAX fare calculator — fare updates instantly when user selects cities and cab type.
- Round trip fare = one-way fare × 1.9.
- Form validation: same pickup/drop city rejected, return date required for round trips.
- On successful booking → auto-generates booking ID like 1313-AB1234 and saves to database.
- Redirects to Booking Success page with full trip summary.

5.3 Fare Calculation Logic

Fixed fare matrix stored in views.py. Works bidirectionally — Chandigarh to Delhi and Delhi to Chandigarh return the same fare:

Route	Sedan	SUV	Tempo Traveller
Chandigarh ↔ Delhi	₹2,800	₹3,800	₹5,500
Chandigarh ↔ Amritsar	₹2,000	₹2,800	₹4,000
Chandigarh ↔ Shimla	₹1,800	₹2,500	₹3,800
Chandigarh ↔ Manali	₹3,500	₹4,800	₹7,000
Chandigarh ↔ Jammu	₹3,000	₹4,200	₹6,000
Chandigarh ↔ Srinagar	₹5,500	₹7,500	₹11,000
Chandigarh ↔ Jaipur	₹5,000	₹6,800	₹9,500

5.4 Live Cab Tracking (track_cab view)

One of the most technically advanced features of the project:

- Customer enters Booking ID and registered phone number to access tracking.
- 10-Minute Window Rule: Live map only activates 10 minutes before pickup time. Before that, a countdown is shown.
- Once active, Google Maps shows the driver's real-time location as a moving yellow marker.
- Map auto-refreshes every 10 seconds via JavaScript setInterval.
- Driver card shows name, vehicle number, and a clickable Call Driver link.
- Status-based messages: Pending → Confirmed → Tracking → On Trip → Completed.

5.5 Routes & Fares Page

- Full table of all 7 routes with distance, time, fare for all 3 cab types.
- Fare notes section: round trip pricing, toll policy, night charges, GST inclusion.

5.6 Contact Page

- Contact form with name, email, phone, message.
- Submitted messages saved to ContactMessage database table.
- Success message shown after submission.

6. Features — Driver / Admin Portal

Accessible at /portal/ — only for drivers and admins. Completely separate from customer website with its own login, layout, and CSS.

6.1 Authentication System

- Login page at /portal/login/ — completely separate from Django admin.
- Uses Django's built-in authenticate() and login() functions.
- Access control: only users who are staff/superuser (admin) OR have a linked Driver object can log in.
- Drivers see driver-specific features; admins see everything.
- Logout at /portal/logout/.

6.2 Live Booking Alert System

This is the real-time notification system for new ride requests:

- Every 10 seconds, the portal JavaScript polls /portal/api/pending/ API.
- If new pending bookings exist → bell icon rings with animation.
- Badge count appears showing number of pending rides.
- Browser notification popup fires (if permission granted).
- Audio beep plays using Web Audio API to alert driver even if screen is elsewhere.
- Alert panel slides open showing all pending rides with customer details and fare.
- First driver to click "Accept" gets the ride — booking status changes to Confirmed.
- Alert automatically disappears for all other drivers on next poll (10 sec).

6.3 Dashboard

- Live stats: Total Bookings, Pending Rides, Completed Rides.
- Admin-only stats: Today's Revenue, Today's Commission, All-time Revenue.
- 7-day revenue bar chart using Chart.js library.
- Pending rides table with Accept button for drivers.
- Driver view: My Recent Rides table with Start Trip and Complete buttons.

6.4 Bookings Management

- Full filterable table of all bookings.

- Filter by: Status, Date, or Search by Booking ID / Name / Phone.
- Driver sees their own rides + all pending rides.
- Admin sees all bookings from all drivers.
- Actions: Accept (driver), Assign Driver (admin), Start Trip, Complete, Cancel.
- Admin can open a modal to manually assign any available driver to a pending booking.
- Commission amount shown for each booking (admin only).

6.5 Driver Location Sharing

How the GPS tracking system works end-to-end:

- Driver opens portal on phone → Dashboard shows green "Start Sharing Location" button.
- On click: browser requests GPS permission via navigator.geolocation API.
- Once granted: GPS coordinates sent to server every 10 seconds via POST to /api/driver/location/update/.
- Server updates Driver.current_lat and Driver.current_lng in database.
- Customer's track page fetches /api/cab/location/<booking_id>/ every 10 seconds.
- Driver marker on Google Maps moves smoothly as coordinates update.
- Driver can click "Stop Sharing" to stop GPS transmission.
- Sharing auto-stops if driver closes the browser tab.

6.6 Drivers List (Admin only)

- Table of all registered drivers with Driver ID, name, phone, vehicle details.
- Shows total completed rides and total earnings per driver.
- Availability status: Available / On Trip / Inactive.

6.7 Revenue & Profits (Admin only)

Complete financial dashboard for the cab company owner:

- Today's Revenue, Commission (15%), and Driver Payout (85%) — at a glance.
- This Month stats: total revenue, commission, ride count.
- All-Time stats: cumulative totals.
- 30-day daily revenue line chart using Chart.js.
- Per-driver breakdown table: each driver's total rides and earnings.

7. API Endpoints

The application exposes several REST-style API endpoints used internally by JavaScript for real-time features:

Endpoint	Method	Access	Description
/api/fare/	GET	Public	Returns fare for given pickup, drop, cab_type params
/api/cab/location/<booking_id>/	GET	Public	Returns current lat/lng of driver assigned to booking
/api/driver/location/update/	POST	Driver (phone)	Driver pushes GPS coordinates to server
/portal/api/pending/	GET	Portal login required	Returns list of all pending bookings for alert system

8. URL Routing

8.1 Customer Website URLs (cabs/urls.py)

URL	View	Page
/	home	Homepage with booking widget
/book/	book_cab	Full booking form
/booking/success/<id>/	booking_success	Confirmation page
/track/	track_cab	Live tracking page
/routes/	routes	All routes and fares
/contact/	contact	Contact form

8.2 Driver Portal URLs (driver_portal/urls.py)

URL	View	Page
/portal/login/	portal_login	Portal login page
/portal/	dashboard	Main dashboard
/portal/bookings/	all_bookings	All bookings table
/portal/bookings/accept/<id>/	accept_booking	Driver accepts ride
/portal/bookings/status/<id>/	update_status	Update booking status
/portal/bookings/assign/<id>/	assign_driver	Admin assigns driver
/portal/drivers/	drivers_list	All drivers (admin)
/portal/revenue/	revenue	Revenue report (admin)
/portal/api/pending/	pending_api	Live alerts polling API

9. End-to-End Workflow

Step 1 — Customer Books a Cab

- Customer visits Rydo.com and fills the booking form.
- System calculates fare from the fare matrix.
- Booking saved to database with status = "pending" and unique ID like 1313-AB1234.
- Customer sees Booking Confirmation page with all trip details.

Step 2 — Driver Gets Alert

- Within 10 seconds, all logged-in drivers see a bell notification.
- Audio beep + browser popup notifies the driver of a new ride.
- Alert shows: customer name, route, fare, pickup date/time.

Step 3 — Driver Accepts

- First driver to click "Accept" gets the booking.
- Booking status changes from "pending" to "confirmed".
- Driver's is_available flag set to False.
- Alert disappears for all other drivers automatically.

Step 4 — Live Tracking

- Driver opens portal on phone → clicks "Start Sharing Location".
- GPS coordinates sent to server every 10 seconds.
- 10 minutes before pickup time, customer's Track page activates Google Maps.
- Customer sees driver's yellow marker moving on the map in real-time.

Step 5 — Trip Completion

- Driver clicks "Start Trip" when they begin driving → status = "on_trip".
- Driver clicks "Complete" when destination reached → status = "completed".
- Driver's is_available flag set back to True automatically.
- Revenue automatically reflected in admin's Revenue dashboard.

10. Setup & Installation

10.1 Prerequisites

Requirement	Version	Check Command
Python	3.8 or higher	python --version
pip	Any recent	pip --version
Django	4.2+	pip show django
SQLite	Built into Python	python -c "import sqlite3"
Browser	Chrome / Edge / Firefox	Any modern browser

10.2 Installation Steps

```
# Step 1: Install Django
pip install django whitenoise
```

```
# Step 2: Create migrations
python manage.py makemigrations cabs
```

```
# Step 3: Apply migrations (creates database tables)
python manage.py migrate
```

```
# Step 4: Create admin account
python manage.py createsuperuser
```

```
# Step 5: Start the server
python manage.py runserver
```

10.3 Adding a Driver

- Go to http://127.0.0.1:8000/admin/
- Auth → Users → Add User → Create username and password for driver
- Cabs → Drivers → Add Driver → Link the user, fill vehicle details
- Driver can now login at http://127.0.0.1:8000/portal/

12. Future Scope & Improvements

Feature	Technology	Priority
Online Payment Gateway	Razorpay API	High
SMS/OTP Verification	Twilio / MSG91	High
Real-time WebSocket Tracking	Django Channels + Redis	Medium
Production Database	PostgreSQL	High (for deployment)
Driver Mobile App	Flutter / React Native	Medium
Customer Rating System	Django models + AJAX	Medium
Route Optimization	Google Directions API	Low
Multi-language Support	Django i18n (Hindi/English)	Low
Email Notifications	Django Email + SMTP	Medium
Analytics Dashboard	Chart.js + more metrics	Low

13. Project Summary

Attribute	Details
Project Name	Rydo — Intercity Cab Booking Platform
Developer	Mantej
Type	Full-Stack Web Application
Backend	Django 4.2 (Python 3.13)
Frontend	HTML5, CSS3, Vanilla JavaScript
Database	SQLite (Django ORM)
External APIs	Google Maps JavaScript API
Total Apps	2 (cabs + driver_portal)
Total Models	3 (Driver, Booking, ContactMessage)
Total Views	15+ (customer + portal + APIs)
Total Templates	13 (7 customer + 6 portal)
Key Features	Real-time GPS tracking, Live booking alerts, Revenue dashboard, Role-based access
Routes Covered	Chandigarh ↔ Delhi, Amritsar, Shimla, Manali, Jammu, Srinagar, Jaipur
Cab Types	Sedan (4), SUV (6), Tempo Traveller (12)
Business Model	15% platform commission, 85% driver payout

— End of Documentation —